

Intel 8085 Microprocessor Simulator

Lab Trainer Kit

Complete Reference Manual

Instruction Set • Assembler Directives • Example Programs

Author: gojonichan
Platform: Python 3.x + Tkinter
Source: <https://github.com/gojonichan/8085Simulator>

September 14, 2026

Contents

1	Introduction	2
1.1	Features	2
1.2	Requirements	2
1.3	How to Run	2
2	Usage Guide	2
2.1	Writing Your First Program	2
2.2	Basic Program Structure	3
2.3	Trainer Kit Mode	3
2.4	Keyboard Shortcuts	3
3	Registers & Flags	3
3.1	8-bit Registers	4
3.2	16-bit Register Pairs	4
3.3	Flags	4
4	Number Formats	4
5	Complete Instruction Set Reference	5
5.1	Data Transfer Instructions	5
5.2	Arithmetic Instructions	5
5.3	Logical Instructions	6
5.4	Branch Instructions	6
5.5	Stack & I/O Instructions	7
5.6	Control Instructions	7
6	Assembler Directives	8
6.1	Labels	8
7	Example Programs	8
7.1	Program 1: Add Two Numbers	8
7.2	Program 2: Find Smallest Number in an Array	9
7.3	Program 3: Bubble Sort (Ascending)	9
7.4	Program 4: 8-bit Multiplication (Repeated Addition)	10
7.5	Program 5: Subroutine Call — Sum of Array	11
7.6	Program 6: I/O Port Demo	11
8	Tips & Common Patterns	12
8.1	Storing an Array in Memory	12
8.2	Looping Through an Array	12
8.3	Comparing Two Numbers	12
8.4	Using the Stack	12
9	Credits	13

1 Introduction

A comprehensive simulator with a graphical user interface that mimics a real 8085 trainer kit. Features a complete instruction set, a two-pass assembler, step execution, breakpoints, and a memory viewer.

1.1 Features

- **Complete 8085 Instruction Set** — Supports all 246 opcodes.
- **Two-pass Assembler** — Labels and directives (**ORG**, **EQU**, **DB**, **DW**, **DS**, **END**).
- **Execution Modes** — Step-by-step execution or continuous run with adjustable speed.
- **Lab Trainer Kit Interface** — Hex keypad and 7-segment display.
- **Memory Viewer & Editor** — View and edit all 64KB of memory directly.
- **I/O Port Simulation** — Interactive I/O port reading and writing.
- **Syntax-highlighted Editor** — Built-in assembly editor with line numbers.
- **Sample Programs** — Over 12 built-in programs (Fibonacci, Bubble Sort, BCD Addition, etc.).

1.2 Requirements

- Python 3.x
- `tkinter` (usually included with standard Python installations)

1.3 How to Run

Clone the repository and run the script:

```
python simulator.py
```

2 Usage Guide

2.1 Writing Your First Program

1. Open the simulator and type your assembly code in the **Editor** tab.
2. Click **Assemble (F8)** to compile. Check the **Console** tab for errors.
3. Click **Run (F5)** to execute continuously, or **Step (F6)** to execute one instruction at a time.
4. View results in the **Registers** panel, **Flags** panel, or **Memory Viewer**.

2.2 Basic Program Structure

Every program typically follows this pattern:

```
; Comments start with a semicolon
ORG 2000H                ; Set the starting address of the
                        program

; Your instructions go here
MVI A, 25H              ; Load a value into accumulator
STA 3000H               ; Store result at memory address 3000H
HLT                    ; Stop execution

; Data section
ORG 2050H               ; Set the address for data
DB 05H, 0AH, 0FH       ; Define bytes in memory
```

2.3 Trainer Kit Mode

Use the **hex keypad** to enter addresses and data directly into memory, just like a real hardware trainer kit:

1. Enter the memory address using hex keys.
2. Press the appropriate function key to examine/modify memory.
3. Use the 7-segment display to view addresses and data values.

2.4 Keyboard Shortcuts

Shortcut	Action
F8	Assemble
F5	Run
F6	Step (one instruction)
F7	Stop
F9	Reset CPU
Ctrl+N	New File
Ctrl+O	Open File
Ctrl+S	Save File

3 Registers & Flags

3.1 8-bit Registers

Register	Description
A	Accumulator — primary register for arithmetic/logic operations
B	General purpose register (pairs with C)
C	General purpose register (pairs with B)
D	General purpose register (pairs with E)
E	General purpose register (pairs with D)
H	General purpose register (pairs with L, forms memory address)
L	General purpose register (pairs with H, forms memory address)
M	Not a real register — refers to memory at address in HL

3.2 16-bit Register Pairs

Pair	Registers	Usage
B	B and C	General purpose pair
D	D and E	General purpose pair
H	H and L	Memory pointer pair
SP	—	Stack Pointer (16-bit)
PSW	A + Flags	Program Status Word (used with PUSH/POP)

3.3 Flags

The 8085 has 5 status flags that are set or cleared based on the result of operations:

Flag	Name	Set When (= 1)
S	Sign	Bit 7 of the result is 1 (result is negative)
Z	Zero	Result is zero
AC	Auxiliary Carry	There is a carry from bit 3 to bit 4
P	Parity	Result has an even number of 1-bits
CY	Carry	There is a carry or borrow from bit 7

4 Number Formats

The assembler accepts numbers in three formats:

Format	Syntax	Example
Hexadecimal	Suffix H or prefix 0x	0FFH, 2000H, 0x2000
Decimal	Plain number	255, 100
Binary	Suffix B	11111111B, 1010B

Note

Hex numbers starting with a letter (A–F) must be prefixed with 0. For example, write 0FFH not FFH.

5 Complete Instruction Set Reference

5.1 Data Transfer Instructions

These instructions move data between registers, memory, and I/O ports **without modifying flags**.

Instruction	Operation	Description
MOV r1, r2	$r1 \leftarrow r2$	Copy register r2 to r1
MVI r, data	$r \leftarrow \text{data}$	Load immediate 8-bit data to register
LDA addr	$A \leftarrow [\text{addr}]$	Load accumulator from memory address
STA addr	$[\text{addr}] \leftarrow A$	Store accumulator to memory address
LHLD addr	$L \leftarrow [\text{addr}], H \leftarrow [\text{addr}+1]$	Load HL pair from memory
SHLD addr	$[\text{addr}] \leftarrow L, [\text{addr}+1] \leftarrow H$	Store HL pair to memory
LXI rp, d16	$rp \leftarrow \text{d16}$	Load 16-bit immediate to register pair
LDAX rp	$A \leftarrow [rp]$	Load A from address in rp (B or D only)
STAX rp	$[rp] \leftarrow A$	Store A to address in rp (B or D only)
XCHG	$DE \leftrightarrow HL$	Exchange DE and HL pairs

5.2 Arithmetic Instructions

These instructions perform addition, subtraction, and increment/decrement. They affect all flags unless noted.

Instruction	Operation	Description
ADD r	$A \leftarrow A + r$	Add register to A
ADC r	$A \leftarrow A + r + CY$	Add register to A with carry
SUB r	$A \leftarrow A - r$	Subtract register from A
SBB r	$A \leftarrow A - r - CY$	Subtract register from A with borrow
INR r	$r \leftarrow r + 1$	Increment register by 1
DCR r	$r \leftarrow r - 1$	Decrement register by 1
INX rp	$rp \leftarrow rp + 1$	Increment register pair (no flags affected)
DCX rp	$rp \leftarrow rp - 1$	Decrement register pair (no flags affected)

Instruction	Operation	Description
DAD rp	$HL \leftarrow HL + rp$	Add register pair to HL (only CY affected)
DAA	Decimal adjust A	Adjust A for BCD addition
ADI data	$A \leftarrow A + data$	Add immediate to A
ACI data	$A \leftarrow A + data + CY$	Add immediate to A with carry
SUI data	$A \leftarrow A - data$	Subtract immediate from A
SBI data	$A \leftarrow A - data - CY$	Subtract immediate from A with borrow

5.3 Logical Instructions

These instructions perform bitwise operations, comparisons, and rotations.

Instruction	Operation	Description
ANA r	$A \leftarrow A \text{ AND } r$	Logical AND register with A
XRA r	$A \leftarrow A \text{ XOR } r$	Logical XOR register with A
ORA r	$A \leftarrow A \text{ OR } r$	Logical OR register with A
CMP r	Compare A with r	Sets flags based on $A - r$, A unchanged
ANI data	$A \leftarrow A \text{ AND } data$	AND immediate with A
XRI data	$A \leftarrow A \text{ XOR } data$	XOR immediate with A
ORI data	$A \leftarrow A \text{ OR } data$	OR immediate with A
CPI data	Compare A with data	Sets flags based on $A - data$, A unchanged
RLC	Rotate A left	Bit 7 $\rightarrow$ CY and bit 0
RRC	Rotate A right	Bit 0 $\rightarrow$ CY and bit 7
RAL	Rotate A left through carry	9-bit: bit 7 $\rightarrow$ CY, old CY $\rightarrow$ bit 0
RAR	Rotate A right through carry	9-bit: bit 0 $\rightarrow$ CY, old CY $\rightarrow$ bit 7
CMA	$A \leftarrow \text{NOT } A$	1's complement of A
CMC	$CY \leftarrow \text{NOT } CY$	Complement carry flag
STC	$CY \leftarrow 1$	Set carry flag

5.4 Branch Instructions

These instructions change the flow of execution. Conditional branches check flag values.

Instruction	Condition	Description
JMP addr	—	Jump unconditionally
JC addr	$CY = 1$	Jump if carry

Instruction	Condition	Description
JNC addr	CY = 0	Jump if no carry
JZ addr	Z = 1	Jump if zero
JNZ addr	Z = 0	Jump if not zero
JP addr	S = 0	Jump if positive
JM addr	S = 1	Jump if minus (negative)
JPE addr	P = 1	Jump if parity even
JPO addr	P = 0	Jump if parity odd
CALL addr	—	Call subroutine (push PC, jump to addr)
RET	—	Return from subroutine (pop PC)
PCHL	—	Jump to address in HL (PC ← HL)
RST n	—	Restart: call address $n \times 8$ ($n = 0-7$)

Conditional CALL variants: CC, CNC, CZ, CNZ, CP, CM, CPE, CPO

Conditional RET variants: RC, RNC, RZ, RNZ, RP, RM, RPE, RPO

5.5 Stack & I/O Instructions

Instruction	Operation	Description
PUSH rp	Stack ← rp	Push register pair onto stack
POP rp	rp ← Stack	Pop register pair from stack
XTHL	(SP) ↔ HL	Exchange top of stack with HL
SPHL	SP ← HL	Load SP from HL
IN port	A ← port	Read from input port into A
OUT port	port ← A	Write A to output port

Note

rp can be B (BC), D (DE), H (HL), or PSW (A + Flags).

5.6 Control Instructions

Instruction	Description
NOP	No operation (does nothing, wastes 1 cycle)
HLT	Halt processor — stops execution
EI	Enable interrupts
DI	Disable interrupts
RIM	Read interrupt mask into A
SIM	Set interrupt mask from A

6 Assembler Directives

Directives are commands to the assembler (not the CPU). They control where code and data are placed in memory.

Directive	Example	Description
ORG addr	ORG 2000H	Set the origin — next instruction/data goes at this address
DB data	DB 05H, 0AH, 0FH	Define byte(s) — store raw data in memory
DW data	DW 2050H	Define word — store 16-bit value (low byte first)
DS count	DS 10H	Define storage — reserve count bytes of memory
EQU value	COUNT EQU 05H	Equate — assign a constant value to a label
END	END	End of source code

6.1 Labels

Labels mark addresses in your program for jumps, calls, and data references:

```

LOOP:                                ; Define a label
    DCR C                            ; Decrement counter
    JNZ LOOP                         ; Jump back to LOOP if not zero

```

- Labels end with a colon (:).
- They can contain letters, digits, and underscores.
- They must start with a letter or underscore.

7 Example Programs

7.1 Program 1: Add Two Numbers

Adds 25H and 1AH, stores the result at address 3000H.

```

; Adds 25H and 1AH, stores result at 3000H
    ORG 2000H

    MVI A, 25H                       ; Load first number into A
    MVI B, 1AH                       ; Load second number into B
    ADD B                            ; A = A + B = 3FH
    STA 3000H                        ; Store result at 3000H
    HLT

```

7.2 Program 2: Find Smallest Number in an Array

Array count is stored at 2050H, data starts from 2051H. The smallest value is stored at 2060H.

```

; Array count at 2050H, data from 2051H
; Result (smallest) stored at 2060H
    ORG 2000H

    LDA 2050H           ; Load the count
    DCR A               ; N-1 comparisons needed
    MOV C, A            ; C = loop counter
    LXI H, 2051H        ; HL points to first element
    MOV A, M            ; A = first element (assume smallest)

FIND:
    INX H               ; Point to next element
    CMP M               ; Compare A with current element
    JC SKIP             ; If A < element, keep A (carry set)
    MOV A, M            ; New smallest found
SKIP:
    DCR C               ; Decrement counter
    JNZ FIND            ; Repeat
    STA 2060H           ; Store smallest at 2060H
    HLT

; Test data
    ORG 2050H
    DB 05H              ; Count = 5
    DB 34H, 78H, 12H, 9AH, 56H ; Array data

```

7.3 Program 3: Bubble Sort (Ascending)

Sorts N bytes starting at 2051H. The count is at 2050H.

```

; Sorts N bytes starting at 2051H, count at 2050H
    ORG 2000H

    LDA 2050H           ; Load count N
    DCR A               ; N-1 passes needed
    MOV C, A            ; C = outer loop counter

OUTER:
    MOV D, C            ; D = inner loop counter
    LXI H, 2051H        ; Point to start of array

INNER:
    MOV A, M            ; Load current element
    INX H               ; Point to next element
    CMP M               ; Compare with next element

```

```

    JC NOSWAP          ; If current < next, no swap
    JZ NOSWAP          ; If equal, no swap

    MOV B, M           ; B = next element
    MOV M, A           ; Store current in next position
    DCX H
    MOV M, B           ; Store next in current position
    INX H

NOSWAP:
    DCR D
    JNZ INNER
    DCR C
    JNZ OUTER
    HLT

; Test data
    ORG 2050H
    DB 05H              ; Count = 5
    DB 64H, 19H, 2DH, 07H, 48H ; Array: 100, 25, 45, 7, 72

```

7.4 Program 4: 8-bit Multiplication (Repeated Addition)

Multiplies two 8-bit values at 2050H and 2051H. The 16-bit result is stored at 2052H–2053H.

```

; Multiplies values at 2050H and 2051H
; 16-bit result stored at 2052H-2053H
    ORG 2000H

    LDA 2050H          ; Load multiplicand
    MOV B, A
    LDA 2051H          ; Load multiplier
    MOV C, A
    MVI A, 00H         ; Clear accumulator
    MVI D, 00H         ; Clear D for carry

LOOP:
    ADD B              ; Add multiplicand
    JNC NOCARRY
    INR D              ; Increment high byte on overflow
NOCARRY:
    DCR C              ; Decrement counter
    JNZ LOOP

    STA 2052H          ; Store low byte of result
    MOV A, D
    STA 2053H          ; Store high byte of result
    HLT

```

```

; Data
  ORG 2050H
  DB 05H           ; Multiplicand = 5
  DB 04H           ; Multiplier = 4

```

7.5 Program 5: Subroutine Call — Sum of Array

Sums 5 bytes at 2050H using a subroutine. Result stored at 2060H.

```

; Sums 5 bytes at 2050H using a subroutine, result at 2060H
  ORG 2000H

  LXI SP, 20FFH      ; Initialize stack pointer
  LXI H, 2050H       ; Array address
  MVI C, 05H         ; Count

  CALL SUM           ; Call subroutine
  STA 2060H          ; Store the sum
  HLT

; Subroutine: SUM
; Input:  HL = array address, C = count
; Output: A = sum
SUM:
  MVI A, 00H         ; Clear accumulator
ADDNXT:
  ADD M              ; Add array element
  INX H              ; Next element
  DCR C
  JNZ ADDNXT
  RET                ; Return to caller

; Data
  ORG 2050H
  DB 10H, 20H, 30H, 40H, 50H

```

7.6 Program 6: I/O Port Demo

Reads from input port 01H, rotates the value left twice, and writes to output port 02H.

```

; Reads from port 01H, rotates left twice, outputs to port 02H
  ORG 2000H

  IN 01H            ; Read from input port 1
  RLC               ; Rotate left
  RLC               ; Rotate left again
  OUT 02H           ; Output to port 2
  HLT

```

8 Tips & Common Patterns

8.1 Storing an Array in Memory

Use **ORG** to set the address and **DB** to define the data:

```
ORG 2050H
DB 05H           ; Count (optional, but useful)
DB 11H, 22H, 33H, 44H, 55H ; Array elements
```

8.2 Looping Through an Array

Use **HL** as a pointer and **C** as a counter:

```
LXI H, 2051H      ; Point HL to first element
MVI C, 05H        ; Load count

LOOP:
MOV A, M          ; Load current element into A
; ... do something with A ...
INX H             ; Move to next element
DCR C             ; Decrement counter
JNZ LOOP          ; Repeat if not zero
```

8.3 Comparing Two Numbers

CMP subtracts without changing **A** — it only sets flags:

```
MOV A, M          ; Load first value
INX H
CMP M             ; Compare A with next value
JC LESS           ; Jump if A < M[HL] (Carry = 1)
JZ EQUAL          ; Jump if A == M[HL] (Zero = 1)
; otherwise A > M[HL]
```

Tip

CMP internally computes **A - operand** and sets flags accordingly, but the accumulator **A** retains its original value. This is essential for search and sort algorithms.

8.4 Using the Stack

Always initialize **SP** before using **PUSH**, **POP**, or **CALL**:

```
LXI SP, 20FFH     ; Stack grows downward from 20FFH
PUSH B            ; Save BC on stack
; ... do work ...
POP B             ; Restore BC
```

Note

The stack in the 8085 grows **downward** — PUSH decrements SP by 2, and POP increments SP by 2. Make sure the stack area does not overlap with your program or data memory.

9 Credits

Developed by **gojonichan** — <https://github.com/gojonichan/8085Simulator>